FINAL YEAR PROJECT

Haven.AI

Complete Project Documentation — AI-Assisted Housing & Real-Estate Platform

Presentation & viva reference: features, technology choices, architecture, security, AI/ML, API, testing and Q&A.

Mian Abdur Rauf · 10 October 2026

Haven.AI — Complete Project Documentation

> **Final Year Project — AI-Assisted Housing & Real-Estate Platform** A single study-and-reference document for the presentation and viva. Written against the actual code in this repository (not the early planning drafts), so everything here is defensible in front of an examiner.

Table of contents

1. What the project is (elevator pitch)
2. The problem it solves and why it exists
3. The seven features — what each does and why it is included
4. Technology stack — what and *why* each was chosen
5. System architecture (how the pieces fit)
6. Module-by-module deep dive
7. Security model
8. Data model (Firestore collections)
9. Full API reference
10. AI / ML explained simply
11. Testing strategy
12. How to run it (demo checklist)
13. Honest limitations & planned-vs-built notes
14. Likely viva questions with model answers
15. 60-second and 5-minute presentation scripts

---

1. What the project is

Haven.AI is a full-stack web application that brings the whole early housing journey into one place. Instead of jumping between a property portal, a drafting tool, a 3D package, an interior-design app and a contractor's quote, a user does all of it in one site:

- **ArchiBot** — an AI chatbot that answers real-estate/architecture questions and routes specialised requests to the right module.
- **2D floor-plan generation** from a plain-language brief.
- **3D house model generation** from a brief, a 2D plan, a plan image, or a property photo — viewable in the browser.
- **Interior design** on the user's *own* room photo — move, recolour, swap and add real furniture, then render a photorealistic image.
- **Construction cost estimation** from a description, with a line-item breakdown.
- **Property listings** (local + optional Zameen.com feed) with buyer ↔ dealer chat.
- **Admin dashboard** to monitor and moderate the platform.

One-line summary for the panel: *"Haven.AI is a unified, AI-assisted real-estate platform — chat, 2D/3D design, interior design on your own photo, cost estimation and verified listings with dealer messaging — built with a React*

front end, a FastAPI back end, Firebase for identity and data, and a mix of cloud AI (Gemini) and on-device models (SegFormer, XGBoost, OpenCV)."

---

2. The problem and why

The problem. Deciding on, designing and budgeting a home today means using many disconnected tools. A buyer searches one portal, sketches a layout in another, pays a designer for interiors, and chases a contractor for a rough cost. The information never lives in one place, and none of it is guided.

The goal. Reduce that complexity by *centralising* these services behind a single, AI-guided interface, aimed at the Pakistani residential market (costs in PKR, areas in *marla/kanal*, cities like Lahore/Karachi/Islamabad, optional Zameen.com listings).

Why it's a good FYP. It is broad enough to demonstrate full-stack engineering (auth, database, security rules, REST API, SPA), *and* deep enough to demonstrate several distinct AI/ML techniques: an LLM-driven chatbot, vision models, classical computer vision, a trained regression model, and real-time 3D graphics — each chosen deliberately rather than "LLM for everything."

---

3. The features

Each feature below lists **what it does** and **why it's in the product**.

#	Feature	What it does	Why it's included
1	**ArchiBot chatbot**	Chat UI that understands a question, routes it to a module, and replies in natural Markdown.	A single front door for everything; lowers the learning curve — users just *ask*.
2	**2D floor-plan generator**	Turns "3-bed 5-marla single-storey house" into a validated 2D plan.	Early-stage planning without a draughtsman; feeds the 3D module.
3	**3D model generator**	Builds a 3D house (glTF/GLB) from a brief, a 2D plan, a plan image, or a property photo.	Lets users *see* a design in 3D in the browser — the "wow" feature.
4	**Interior design on your photo**	Upload a real room photo; move/resize/recolour/replace/add real furniture; export a photorealistic render.	Most interior apps use generic rooms; using the user's *own* photo is the differentiator.
5	**Cost estimation**	Natural-language brief → total cost, per-sqft rate, CSI-style line items, and saving tips.	The single most-asked real-estate question: "what will it cost?"
6	**Property listings + dealer chat**	Browse/search local + Zameen listings; buyers message dealers; dealers manage their own listings.	Turns the tool from a sandbox into a two-sided marketplace.
7	**Admin dashboard**	Platform stats, user management, listing verification.	Governance/moderation — required for any real multi-user platform.

---

4. Technology stack and why

This is the section examiners probe hardest. For each choice there is a **reason** and an **alternative we rejected**.

Frontend

Technology	Version	Why chosen	Rejected alternative
React	19	Component model suits a multi-feature SPA; huge ecosystem; the team's strongest skill.	Angular/Vue — heavier / smaller ecosystem for our needs.
Vite	7	Instant dev server + fast builds; simple `/api` dev proxy to the backend.	Create-React-App — deprecated, slow.
React Router	7	Client-side routing with lazy-loaded pages (first visit downloads only what it shows).	—
Three.js + @react-three/fiber	0.186 / 9	Industry-standard WebGL in the browser; `fiber` lets us drive Three.js declaratively from React. Renders the generated `.glb` houses.	Babylon.js — comparable, but Three.js has more learning material.
Firebase JS SDK	12	Auth + direct Firestore reads/writes from the browser (listings, profiles, buyer↔dealer chat) with no backend round-trip.	Rolling our own auth — insecure and time-consuming.
Plain CSS (`index.css`)	—	One hand-written design system; no build-time CSS framework needed.	*(The early PRD mentioned Tailwind; we did not use it — see §13.)*
Vitest + Testing Library	3	Fast, Vite-native unit tests for components and pure helpers.	Jest — slower to configure with Vite.

Backend

Technology	Version	Why chosen	Rejected alternative
FastAPI	0.135	Async Python, automatic OpenAPI docs at `/docs`, Pydantic validation; ideal for wrapping ML/AI services behind clean REST.	Flask — no async / built-in validation; Django — too heavy for an API-only backend.
Uvicorn	0.41	ASGI server to run FastAPI's async endpoints.	—
Pydantic / pydantic-settings	2.x	Request/response validation and typed settings loaded from `.env`.	—
Firebase Admin SDK	7.3	Verifies the browser's ID tokens, manages the `admin` custom claim, and writes backend-only collections (not subject to client rules).	—
google-genai (Gemini)	2.17	Cloud LLM + vision for chat, 2D layout, plan reading and photo analysis. Free-tier *flash* models keep it zero-cost for an FYP.	OpenAI — paid; self-hosted LLM — needs a GPU we don't have. *(Early docs said "Ollama"; we used Gemini — see §13.)*

AI / ML & vision (why a *mix*, not one model)

Technology	Used for	Why this and not an LLM
Gemini flash (cloud)	Chatbot phrasing, 2D layout JSON, plan-image reading, room-photo analysis.	Best for open-ended language and multimodal understanding.
SegFormer-B2 on ONNX Runtime (on-device, CPU)	Interior: segmenting a room photo into walls/floor/furniture.	A fast, free, offline vision model; ~2 s/photo, no API key, no per-call cost, and it keeps the user's photo private on the server.
XGBoost + scikit-learn (trained model)	Cost estimation (hybrid rule + ML).	Costs are a *structured regression* problem — a trained gradient-boosted model is far more accurate and explainable than asking an LLM to guess a number.
OpenCV (headless) + RapidOCR	Classical floor-plan image analysis & reading printed dimensions.	Deterministic geometry/OCR; cheaper and more reliable than vision AI for ruled drawings.
Three.js geometry / glTF	Deterministic 3D house construction and real-time rendering.	The house is *built* by rules from validated data, so it's always physically sensible — not hallucinated.

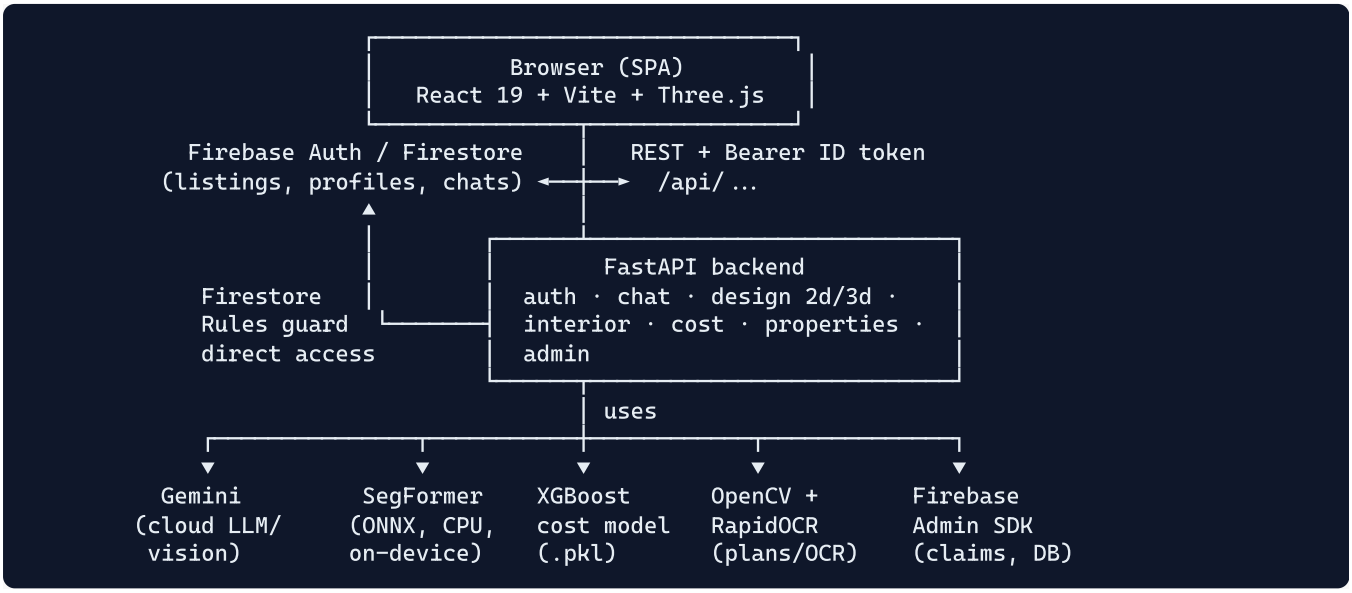
Key design principle to state in the viva: "*We use the right tool for each job. LLMs where language/ambiguity dominates; trained models and classical CV where accuracy, determinism and cost matter; and local models where privacy and zero-cost offline operation matter.*"

Data & infrastructure

Technology	Why
Firestore (NoSQL)	Realtime listeners power buyer↔dealer chat with no extra server; flexible schema for listings/profiles; scales without us managing a DB server.
Firestore Security Rules	The browser talks to Firestore *directly*, so these rules are the real access control for profiles, listings and chats.
`.env` configuration	All secrets/toggles (Gemini key, SMTP, Zameen, feature switches) outside code; template in `.env.example`.
SMTP (e.g. Gmail App Password)	Emails the 6-digit password-reset code.

---

5. System architecture



Two paths to data — this is the single most important architectural point:

1. **Direct browser → Firestore** for *listings, user profiles and buyer↔dealer chat*. Fast, realtime, no backend needed — **protected by** `firestore.rules`.
2. **Browser → FastAPI → services** for *everything that needs a secret, heavy compute, or an AI model* (chat, 2D/3D, interior, cost, admin). The backend holds the Gemini key, runs the models, and uses the Admin SDK to touch backend-only collections.

Why split it this way? Realtime chat and public listings are cheapest and simplest done directly against Firestore; anything involving an API key, a model, or privileged writes must go through the server so secrets and authorisation never reach the browser.

Request lifecycle (example: a chat message)

1. User types in the chat; `api.js` attaches the Firebase **ID token**.
2. `POST /api/chat/send` → `auth.py` verifies the token (or treats as guest).
3. `rate_limit("chat")` checks the per-user/IP quota.
4. `routing.classify_intent` keyword-matches to one of 5 modules (or `general_chat`).
5. If a module matches, `call_module_api` runs it (cost/property/etc.).
6. `orchestrator.generate_llm_response` asks Gemini to phrase the module's data nicely (falling back to a template if Gemini is down).
7. The exchange is cached and saved to `chat_sessions/{id}` in Firestore.

---

6. Module deep dive

6.1 ArchiBot chatbot (`app/services/chat/`)

- **Routing** (`routing.py`): a fast, deterministic **keyword classifier** maps the message to `design_2d`, `design_3d`, `interior_design`, `cost_estimation`, `property_rec`, or falls through to `general_chat`. It also pulls out entities (location, property type, BHK, budget, area) with regex so downstream modules get structured params. *Why keyword-first?* It's instant, free, and predictable; the LLM only handles the open-ended `general_chat` case and the final phrasing.

- **Orchestrator** (`orchestrator.py`): calls the matched module, then asks Gemini to present the result. **Prompt-injection defence:** the system prompt goes through Gemini's `system_instruction` channel and the user's text is wrapped in a `<user_message>` block that is explicitly "data, not instructions." The model is told never to fabricate figures.
- **Resilience:** on a Gemini 429/quota error or any failure it returns a **template response** built from the module data, so the bot never hard-fails.
- **Memory:** last 6 turns are sent for context so follow-ups ("what about a bigger one?") resolve correctly. History is stored in Firestore keyed by the verified uid (guests get a random browser id — never a user id from the body).
- **Caching** (`cache.py`): identical (session, message) pairs return instantly.

6.2 2D floor-plan generation (`app/services/design2d/`)

- Brief → Gemini proposes a layout → **deterministic validator** (`validator.py`) checks room counts, adjacencies and bounds before it's accepted. *Why validate?* The LLM can propose; the validator guarantees the output is structurally sane rather than a hallucinated drawing.

6.3 3D model generation (`app/services/design3d/`)

Four entry points, one deterministic builder: - `POST /generate` — from a **text brief** (`DESIGN3D_TEXT_GENERATOR` : local rule-based planner, or Gemini proposes the 2D layout which is then built). - `POST /from-2d` — from an existing **2D plan**. - `POST /generate-from-image` — from a **property photo** (`DESIGN3D_PHOTO_ANALYZER`). - `POST /analyze` — **reads a floor-plan image** (`PLAN_ANALYZER` : local OpenCV pipeline or Gemini vision).

Pipeline (`service.py`): `prompt → requirement extraction → plot validation (setbacks, max storeys) → space planning → feasibility → 3D geometry → geometry validation → model` . The result is exported as **glTF/GLB** (`gltf_export.py`) and served at `GET /api/design/3d/models/{id}.glb` , then rendered in the browser with Three.js. *Why deterministic geometry?* A house must obey setbacks, wall heights and non-overlapping rooms — rules, not guesses. The AI helps *plan*; the geometry is *computed*, so it's always buildable-looking.

6.4 Interior design (`app/services/interior/`)

The flagship module. Flow: `upload → validate → segment (SegFormer, the only AI step) → build editable scene → edit (move/resize/rotate/recolour/re-material/replace/add/remove) with undo/redo → render → final photorealistic image (PNG/JPEG)` . - **Segmentation**: SegFormer-B2 (ADE20K) via ONNX Runtime on CPU (~2 s, no API key). Any model satisfying the `segment()` contract can replace it. - **Rendering**: real glTF furniture placed through the photo's estimated camera, rasterised with a z-buffer and shaded with **GGX PBR** under lighting estimated from the photo itself (key light from a visible window/lamp, bounce from walls/floor, white balance on the walls). Soft shadows + contact occlusion. - **Object library**: pure JSON (`library_data/`). Items are `mesh` (real CC0 3D models from Poly Haven), `parametric` (code-generated), or `image` (upright PNG). Add furniture by editing JSON only; the loader validates at start-up. - **Replace keeps size/place/turn** (`fit.py`) so swaps look natural. - **Privacy:** runs *entirely on the server* — no cloud AI — and the uploaded photo is never modified. Unsaved rooms auto-delete after 24 h. - **Saved designs** → Firestore `interior_designs` (written by the backend after verifying the ID token).

6.5 Cost estimation (`app/services/cost/` + `ml_models/`)

- A **hybrid estimator**: a transparent **rule-based** engine (per-sqft rates × CSI line items: earthwork, concrete/RCC, formwork, rebar, masonry, woodwork, roofing, doors/windows, finishes, plumbing, electrical) blended with an **XGBoost** ML prediction. The response reports each model's estimate, the blend weights

(`alpha_rule` / `alpha_ml`), the disagreement %, a low–high range, and saving tips. *Why hybrid?* The rule engine is explainable and always available; the ML model captures patterns from data. Blending gives accuracy *and* a story you can defend line by line.

- **Graceful degradation:** if `xgboost` / `sklearn` or the trained `.pkl` is missing, it raises `CostModelUnavailable` and the chatbot falls back to a typical-rate estimate. The model is loaded lazily on first use, so the server starts fine without it.

6.6 Properties & dealer chat

- Listings, profiles and chats live in **Firestore, accessed directly by the browser**, guarded by `firestore.rules`.
- Optional **Zameen.com** feed via `GET /api/properties/external` (`app/services/zameen.py`): off / `MOCK` sample listings / real API — the key stays server-side, responses cached ~10 min.
- Buyer↔dealer chat uses deterministic doc ids (`buyerId_dealerId_propertyId`); rules ensure only the two real participants read/write, and the dealer must be the listing's actual dealer.

6.7 Admin (`app/api/endpoints/admin.py`, `app/services/admin.py`)

- Stats, recent activity, user list/delete, listing verification, view-count reset, health. Every route requires the Firebase `admin: true` **custom claim**, granted only with the service account via `scripts/set_admin.py`.

---

7. Security model

Concern	How it's handled
Who is calling	Browser signs in with Firebase Auth → sends `Authorization: Bearer <ID token>`; `core/auth.py` verifies it (`optional_user`, `require_user`, `require_admin`).
Admins	Firebase custom claim `admin: true`, set only with the service account (`scripts/set_admin.py`). All `/api/admin/*` require it.
Direct Firestore access	Locked down by `firestore.rules`: profiles private to owner; listings public-read but owner-only writes (verification/views are protected); chats readable only by the two participants; backend-only collections fully closed.
Chat identity	Signed-in users keyed by verified uid; guests by a random browser id — **never** a user id taken from the request body (prevents impersonation).
Prompt injection	User text passed to Gemini as delimited *data*, with the system role instructing the model to ignore embedded commands.
Abuse / quota	`rate_limit( ... )` per uid/IP on every expensive endpoint (chat, 2D/3D, interior upload/render, cost, password reset). Default 20/min.
Secrets	Gemini key, SMTP creds, service-account JSON all in `.env` / ignored files — never committed.
Errors	Clients get a generic message; details only in the server log.
Password reset	6-digit code emailed over SMTP, rate-limited (request 5/min, verify/confirm 10/min).

---

8. Data model

Firestore collections (NoSQL documents):

Collection	Written by	Contents	Access
`users/{uid}`	Browser (self)	name, cnic, mobile, email, **role** (buyer/seller), createdAt	Private to owner + admin; role/email immutable after creation.
`properties/{id}`	Browser (dealer)	title, price, dealerId, `verified`, `views`, details	Public read; owner edits (not verified/views); anyone bumps `views` by 1; admin sets `verified`.
`chats/{buyerId_dealerId_propertyId}`	Browser (participants)	lastMessage, unread counters	Only the two participants; dealer must own the listing.
`chats/ … /messages/{id}`	Browser (participants)	senderId, text (≤2000), createdAt	Participants only; messages are immutable.
`chat_sessions/{id}` + `messages`	**Backend** (Admin SDK)	ArchiBot history, last module	Closed to the browser entirely.
`interior_designs/{id}`	**Backend** (Admin SDK)	saved interior scenes	Closed to the browser; backend verifies the token.

9. API reference

Base: `/api`. Heavy endpoints are rate-limited. Interactive docs at `/docs`.

Chat — `POST /chat/send`, `GET /chat/status` **Cost** — `POST /cost/estimate` **2D Design** — `POST /design/2d/generate` **3D Design** — `POST /design/3d/generate`, `POST /design/3d/from-2d`, `POST /design/3d/generate-from-image`, `POST /design/3d/analyze`, `GET /design/3d/models/{id}.glb` **Interior** — `POST /interior/scenes`, `GET /interior/scenes/{id}`, `POST /interior/scenes/{id}/operations|undo|redo|reset|final`, `GET /interior/scenes/{id}/render|original`, `GET /interior/scenes/{id}/objects/{oid}/sprite`, `GET /interior/catalog`, `GET /interior/library/categories|search|items/{id}|items/{id}/thumbnail.png|materials/{m}/swatch.png`, `POST/GET/PUT/DELETE /interior/designs[/ {id}]` **Properties** — `GET /properties/external` **Admin** (claim required) — `GET /admin/stats|activity|users|properties|health`, `DELETE /admin/users/{uid}`, `POST /admin/properties/{id}/verify`, `POST /admin/properties/reset-views` **Account** — `POST /auth/password-reset/request|verify|confirm`

10. AI / ML explained

Have a crisp, non-hand-wavy answer for each model:

- **Gemini (LLM + vision, cloud).** *Classification-light, phrasing-heavy.* We use it for natural language (chatbot answers), for proposing 2D layouts as JSON, and for reading/analysing images where language understanding helps. We pin **flash** models because they have free-tier quota; Pro returns 429 on a free key. We never let it invent numbers — module data is the source of truth.
- **SegFormer-B2 (semantic segmentation, on-device).** A transformer vision model trained on ADE20K, exported to **ONNX** and run on CPU with `onnxruntime`. It labels each pixel of a room photo (wall, floor, sofa,

...) so the interior editor knows what's where. Local = free, private, offline. (NVIDIA research/non-commercial licence — fine for academia, swap for commercial use.)

- **XGBoost (gradient-boosted regression).** Trained on construction data to predict cost; blended with a transparent rule engine. Chosen over an LLM because cost is a numeric prediction that must be accurate and explainable.
- **OpenCV + RapidOCR (classical CV).** Deterministic line/room detection and reading printed dimensions off floor-plan images — exact geometry beats a vision model here.
- **Three.js / glTF (graphics, not ML).** Deterministic 3D construction and WebGL rendering; the AI plans, the geometry is computed.

The headline: five different techniques, each matched to the shape of its problem — language, pixels, numbers, lines, and geometry.

---

11. Testing

- **Backend — pytest** (`haven-ai-backend/tests/`): security/rules (`test_security.py`), chatbot (`test_chat.py`), 3D API (`test_api_design3d.py`), interior across all layers (`test_interior_*` : vision, geometry, render, library, model, replace/fit, realism), plan reading (`test_plan_reading.py` , `test_layer1_vision.py` , `test_layer2_geometry.py`), password reset, Zameen. Fixtures include real room photos and sample plans. Run: `python -m pytest` (~4 min; interior-model tests skip if the ONNX model is absent).
- **Frontend — Vitest + Testing Library**: components and pure helpers (`*.test.jsx/.js`) — editor math, plan review, viewer utils, protected routes, signup, properties, markdown rendering, interior dialogs. Run: `npm test` ; lint with `npm run lint` .
- **Philosophy:** every module degrades gracefully (Gemini down → template; no cost model → typical rate; no ONNX → 503 on that one endpoint), and tests cover those fallback paths so a demo never dies on a missing optional dependency.

---

12. How to run it

```
# Backend  → http://localhost:8000  (docs at /docs)
cd haven-ai-backend
python -m venv venv && venv\Scripts\activate
pip install -r requirements-dev.txt
copy .env.example .env          # add GEMINI_API_KEY; add firebase-credentials.json
uvicorn app.main:app --reload

# Frontend → http://localhost:5000  (proxies /api to :8000)
cd haven-ai-react
npm install && npm run dev
```

First-time Firebase: `firebase deploy --only firestore:rules` , then make yourself admin: `python scripts/set_admin.py you@example.com` (sign out/in).

Optional heavy assets (for the full interior demo): - SegFormer ONNX model into `ml_models/segformer-b2-ade/` (else `POST /interior/scenes` → 503). - Poly Haven furniture/materials: `python scripts/fetch_interior_assets.py` (~210 MB; `--high` adds ~660 MB for final renders). - Cost model: trained `xgb_cost_model.pkl` in `ml_models/` (else typical-rate fallback).

Demo tip: set `RESET_CODE_LOG_ONLY=true` to test password reset without SMTP, and `ZAMEEN_API_URL=MOCK` for sample listings.

---

13. Limitations and divergences

Be upfront about these — examiners respect honesty and it heads off "gotcha" questions.

- **Planning docs vs. final build.** The early `docs/PRD.md` and `docs/phases.md` named **Tailwind, PostgreSQL, Ollama and Docker**. The delivered system uses **hand-written CSS, Firebase/Firestore, and Gemini** instead. *Why we changed:* Firebase gave us auth + realtime DB + security rules out of the box (far less to build and secure than Postgres + a custom auth layer); Gemini's free *flash* tier removed the need for a self-hosted Ollama/GPU; and a single CSS design system was lighter than adding Tailwind. These are the planning documents; this file reflects what actually ships.
- **Phase 3 (deeper AI/ML) was deliberately deferred** per the project owner, to keep the delivered scope stable and demoable.
- **Optional dependencies.** Full interior rendering needs the ONNX model and Poly Haven assets; real cost predictions need the trained `.pkl`. Without them the app still runs with graceful fallbacks.
- **Licensing.** SegFormer-B2 weights are NVIDIA *research/non-commercial* — fine for this FYP, must be swapped before any commercial deployment. Furniture/ textures are CC0 (Poly Haven).
- **Gemini free-tier limits.** Flash models only; Pro = 429 on a free key; image generation needs billing enabled, so the photorealistic *render* path is optional.

---

14. Viva Q&A

Q: Why FastAPI over Flask/Django? Async (needed for concurrent AI calls), automatic request validation with Pydantic, and auto-generated OpenAPI docs — ideal for an API-only backend that wraps several ML services.

Q: Why Firebase instead of a SQL database? It bundles authentication, a realtime NoSQL database, and declarative security rules. Buyer↔dealer chat is realtime with zero extra server code, and the browser can read listings directly — the rules enforce access. A SQL stack would have meant building auth, realtime, and an access layer ourselves.

Q: If the browser writes to Firestore directly, how is that secure? `firestore.rules` is the enforcement point: profiles are private to their owner, listings are public-read but owner-only-write (with `verified` / `views` protected), and chats are restricted to the two participants. Anything needing a secret or privilege goes through the backend instead.

Q: Why not use the LLM for everything (cost, segmentation, 3D)? Because each problem has a different shape. Cost is a numeric regression → XGBoost (accurate, explainable). Segmentation is per-pixel labelling → SegFormer. 3D must obey physical rules → deterministic geometry. LLMs are used only where language/ambiguity dominates. This also controls cost and avoids hallucinated numbers.

Q: What happens when Gemini is down or rate-limited? The chatbot catches 429/errors and returns a template response built from the real module data, so it never hard-fails. Cost falls back to typical rates; interior works offline entirely.

Q: How do you stop prompt injection? The system prompt goes through Gemini's `system_instruction` channel; user text is wrapped in a `<user_message>` block declared as data, and the model is instructed to ignore embedded commands and never fabricate figures.

Q: How is the interior render made to look real? Real glTF furniture is placed through the photo's estimated camera, rasterised with a z-buffer, and shaded with GGX physically-based rendering under lighting *estimated from the photo itself* (window/lamp key light, wall/floor bounce, white balance on the walls), with soft shadows and contact occlusion.

Q: How does the chatbot decide which module to use? A fast deterministic keyword classifier (regex) picks one of five modules or falls to general chat; it also extracts entities (location, budget, area, BHK) for the modules. The LLM only handles general chat and final phrasing — cheap and predictable.

Q: Why hybrid cost model, not pure ML? The rule engine is fully explainable (CSI line items) and always available; the ML model adds data-driven accuracy. Blending reports both estimates and their disagreement, so the number is defensible rather than a black box.

Q: Is the user's room photo sent to the cloud? No. Interior design runs entirely on the server with a local model; the photo is never modified and unsaved rooms are deleted after 24 h.

Q: How do admins get their privileges? A Firebase custom claim `admin: true`, set only with the service-account key via `scripts/set_admin.py` — it cannot be granted from the browser. Every `/api/admin/*` route verifies it.

Q: How would this scale / what's next? Firestore and Gemini scale managed; the backend is stateless behind rate limits. Next steps: the deferred Phase-3 AI work, a commercially-licensed segmentation model, and billing-enabled photorealistic generation.

---

15. Presentation scripts

60-second version:

> "Haven.AI unifies the early home journey — searching, designing in 2D and 3D, styling your own room photo, estimating cost, and talking to dealers — behind one AI assistant. React front end, FastAPI back end, Firebase for identity and realtime data. We deliberately mix AI techniques: Gemini for language and vision, a local SegFormer model for private room segmentation, XGBoost for explainable cost prediction, OpenCV for reading floor plans, and Three.js for real-time 3D. Every AI path has a graceful fallback, and security is enforced by Firebase rules plus token-verified backend routes."

5-minute flow: 1. **Problem** (30s) — fragmented tools; we centralise them (§2). 2. **Live demo** (2.5m) — ArchiBot → cost estimate → generate a 2D plan → view it in 3D → upload a room photo and restyle it. 3. **Architecture** (1m) — the two data paths diagram (§5) and why. 4. **AI choices** (45s) — right tool per problem (§10). 5. **Security & robustness** (15s) — rules, tokens, rate limits, fallbacks.

This document reflects the code as of the current branch. For deeper detail see `haven-ai-backend/README.md` (security + interior internals), the module source under `app/services/`, `firestore.rules`, and `docs/development-log.md` for the build history.